



Natural Language Processing

A Comprehensive Journey into NLP Techniques and Applications

Dated: 12 July 2025

Instructor : Fawad Bahadur

Training Objectives & Outcomes



1

Core NLP concepts and terminologies.

2

Techniques for text preprocessing and analysis.

3

Building NLP models using popular libraries.

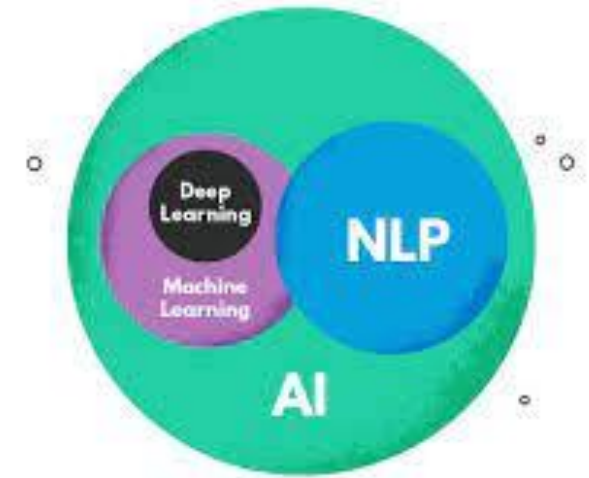
4

Advanced topics like topic modeling and deep learning in NLP.

What is Natural Language Processing (NLP)?

Definition:

Natural Language Processing (NLP) is a field of Artificial Intelligence (AI) that enables computers to **understand**, **interpret**, and **generate** human language.



Goals of NLP:

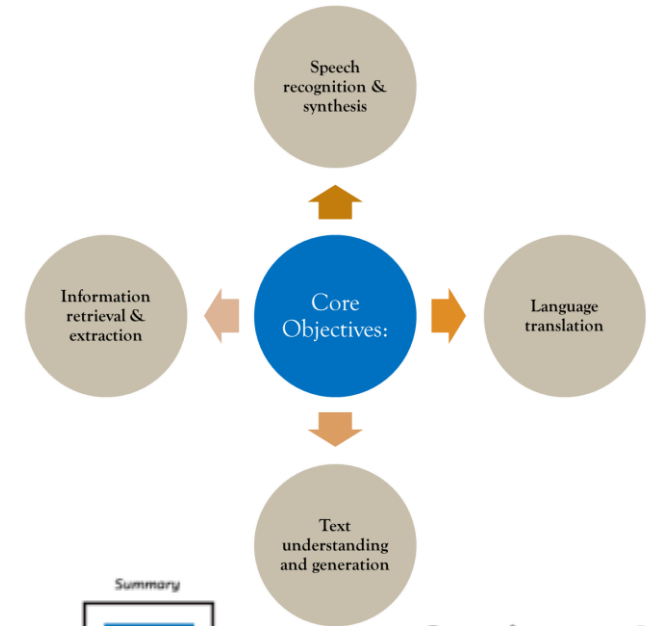
- Bridge communication between humans and machines
- Enable machines to understand context, semantics, and intent
- Automate and enhance language-based tasks



What is Natural Language Processing (NLP)?

Core Objectives:

- Speech recognition & synthesis
- Language translation
- Text understanding and generation
- Information retrieval & extraction

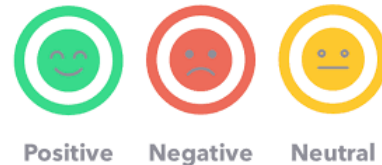


Key Applications:

- Chatbots & Virtual Assistants (e.g., Siri, Alexa)
- Machine Translation (e.g., Google Translate)
- Sentiment Analysis (e.g., product reviews)
- Text Summarization
- Email classification (spam detection)
- Search engines



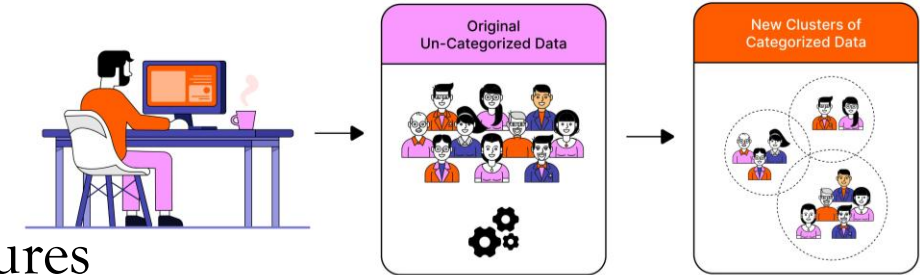
Sentiment Analysis



Brief History of NLP

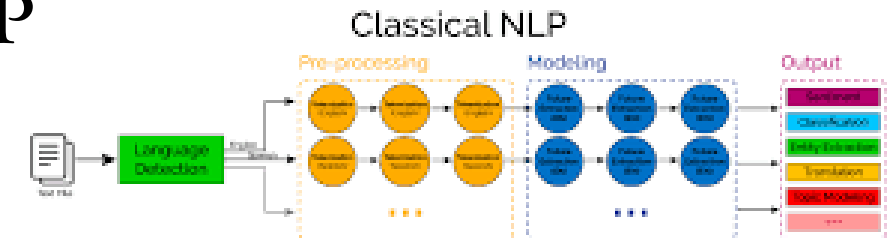
1950s–1960s: Rule-Based Systems

- Alan Turing proposes the Turing Test (1950)
- First attempts at machine translation
- NLP based on hand-written rules and grammar structures



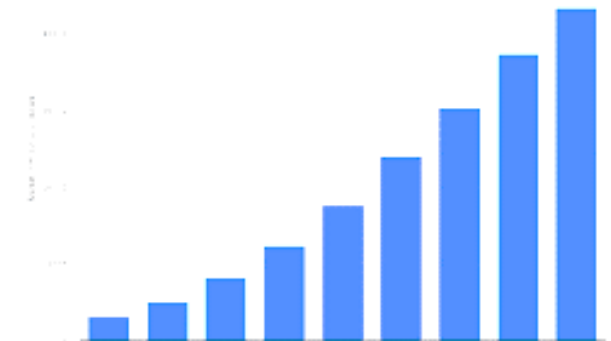
1970s–1980s: Classical Linguistics & Symbolic NLP

- Development of syntactic parsers
- Use of expert systems
- Limited success due to complexity and ambiguity of language



1990s: Statistical NLP

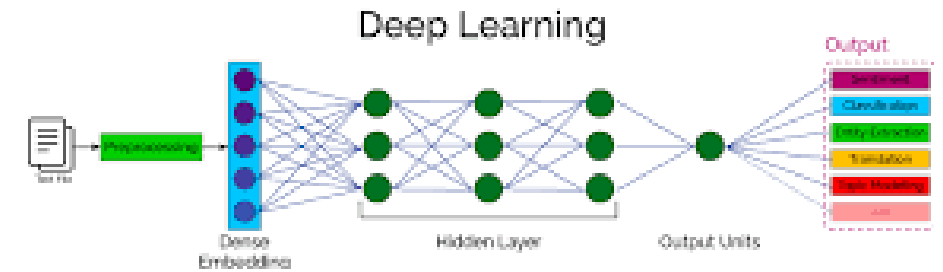
- Introduction of machine learning techniques
- Use of Hidden Markov Models (HMMs) and n-grams
- Rise of data-driven approaches using large corpora



Brief History of NLP

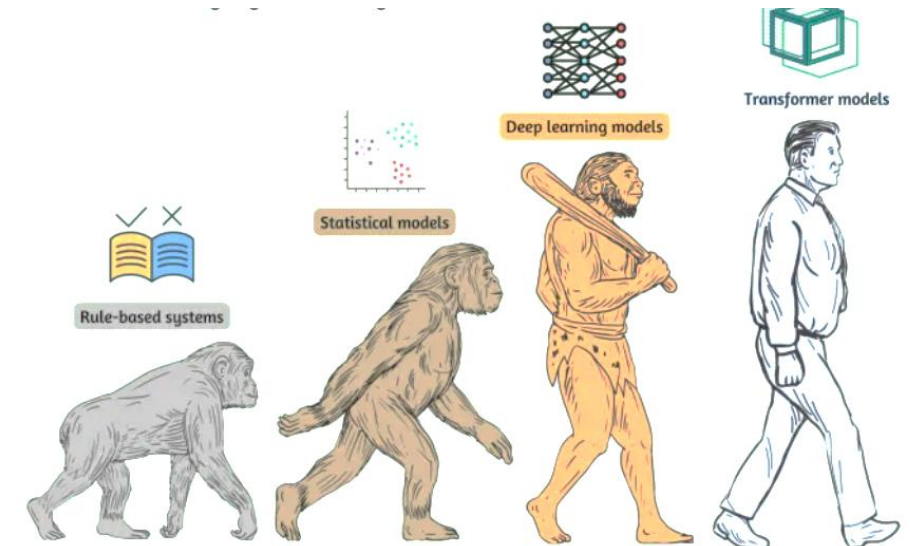
2000s–2010s: Deep Learning Era

- Emergence of word embeddings (e.g., Word2Vec, GloVe)
- NLP powered by Neural Networks and Recurrent Neural Networks (RNNs)
- Significant improvement in accuracy and scale



2018–Present: Transformers and Large Language Models (LLMs)

- Introduction of Transformers (Vaswani et al., 2017)
- Rise of BERT, GPT, T5 and other LLMs
- Shift to pretrained models and transfer learning



Challenges and Ambiguities in Natural Language

1. Ambiguity

Lexical Ambiguity: One word with multiple meanings

E.g., “bat” (animal vs. sports equipment)

Syntactic Ambiguity: Multiple possible sentence structures

E.g., “I saw the man with the telescope”

Semantic Ambiguity: Ambiguity in meaning

E.g., “The chicken is ready to eat” (Who’s eating whom?)



2. Context Sensitivity

- Language meaning depends on context, tone, culture, etc.
 - E.g., sarcasm, irony, or slang



Challenges and Ambiguities in Natural Language

3. Coreference Resolution

Identifying what a pronoun or noun refers to

E.g., “John helped Ali because he was tired”



4. World Knowledge

NLP models lack real-world understanding

E.g., “She put the book on the table and sat on it”



Challenges and Ambiguities in Natural Language

5. Spelling & Grammar Variations

Informal writing, typos, abbreviations

E.g., “u” vs. “you”, “gonna” vs. “going to”



6. Multilinguality & Code-Switching

Handling multiple languages or mixed language text

E.g., “Main office jaa raha hoon” (English + Urdu/Hindi)



The NLP Pipeline – Overview of Common Steps

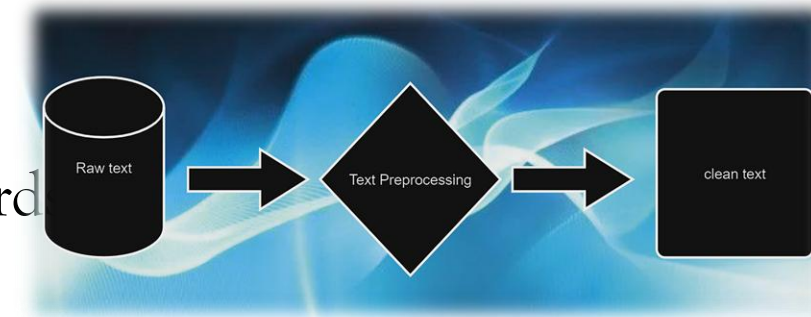
1. Text Collection

Gather raw text data from sources like web, documents, chats, etc.



2. Text Preprocessing

- Tokenization: Splitting text into words/sentences
- Lowercasing: Converting all text to lowercase
- Stopword Removal: Removing common non-informative words
- Stemming/Lemmatization: Reducing words to root forms
- Noise Removal: Removing punctuation, numbers, HTML tags, etc.



The NLP Pipeline – Overview of Common Steps

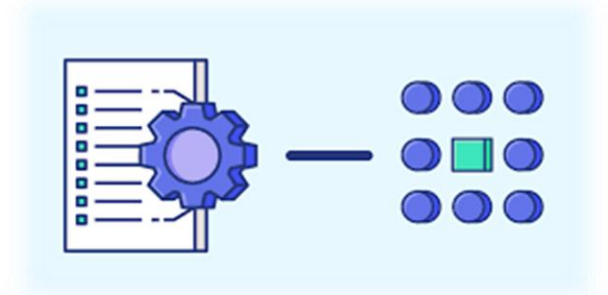
3. Text Representation

- Bag of Words (BoW)
- TF-IDF
- Word Embeddings (Word2Vec, GloVe, BERT)



4. Feature Engineering

Creating features that represent key information from the text



5. Model Building

- Classification (e.g., spam detection)
- Clustering (e.g., topic modeling)
- Sequence Modeling (e.g., language generation)



The NLP Pipeline – Overview of Common Steps

6. Evaluation

- Using metrics like Accuracy, Precision, Recall, F1-Score



7. Deployment

- Integrating trained model into applications or APIs



Introduction to Core NLP Libraries

1. NLTK (Natural Language Toolkit)

- One of the oldest NLP libraries in Python
- Great for learning & prototyping
- Offers tools for tokenization, POS tagging, parsing, and more
- Rich collection of linguistic datasets
- Use Case: Academic projects, initial NLP exploration



Introduction to Core NLP Libraries

2. spaCy

- Fast, production-ready NLP library
- Supports named entity recognition (NER), POS tagging, dependency parsing
- Pre-trained pipelines for multiple languages
- Easy integration with deep learning frameworks
- Use Case: Industrial NLP tasks, pipelines in production



Introduction to Core NLP Libraries

3. Hugging Face Transformers

- Leading library for transformer-based models (BERT, GPT, RoBERTa, etc.)
- Supports fine-tuning and inference with minimal code
- Access to large-scale pre-trained models via transformers & datasets packages
- Use Case: State-of-the-art tasks like text generation, summarization, question answering

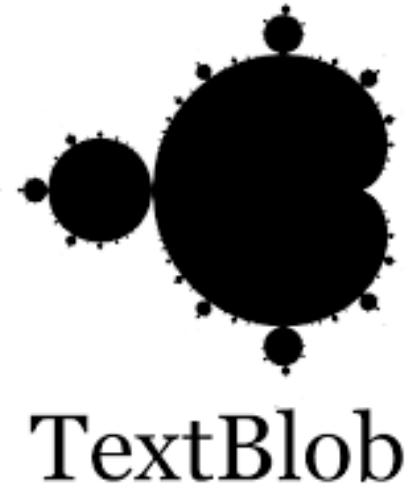


HUGGING FACE

Introduction to Core NLP Libraries

4. Others (Brief Mentions)

- TextBlob: Simple API for common NLP tasks
- Gensim: Topic modeling and Word2Vec
- Flair: NLP by Zalando with strong sequence labeling support



Text Cleaning — Case Folding, Punctuation, Numbers

Objective:

Prepare raw text data for NLP tasks by removing noise and standardizing input.

1. Case Folding (Lowercasing)

- Converts all text to lowercase
- Reduces vocabulary size
- **Example:**
 - "Apple" → "apple"
- Ensures "Apple" and "apple" are treated the same



"She was born in London."



"she was born in london."

Text Cleaning — Case Folding, Punctuation, Numbers

2. Removing Punctuation

- Eliminates special characters like . , ! ? : ; etc.
- Punctuation often does not add meaning (except in sentiment or emotion detection)

Example:

"Hello, world!" → "Hello world"

3. Removing Numbers

- Numbers may be irrelevant in some NLP tasks (e.g., sentiment analysis)
- In other contexts (e.g., financial or scientific texts), they may be retained

Example:

"There are 123 apples" → "There are apples"

Note:

- Always tailor cleaning to your task.
- Don't over-clean if the information might be useful (e.g., "COVID-19", "Windows 10")

Remove punctuation

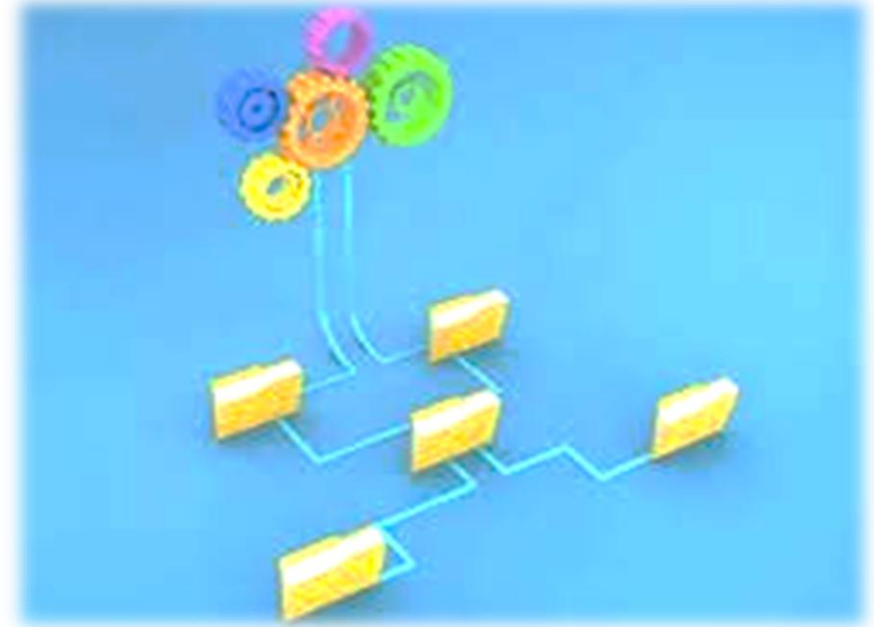
k(/)od/ecli/%k
On#line
A&ca(de@my



Tokenization — Word, Sentence, and Subword Tokenization

Definition:

Tokenization is the process of splitting text into smaller units (tokens), which are the building blocks for NLP tasks.



Tokenization — Word, Sentence, and Subword Tokenization

1. Word Tokenization

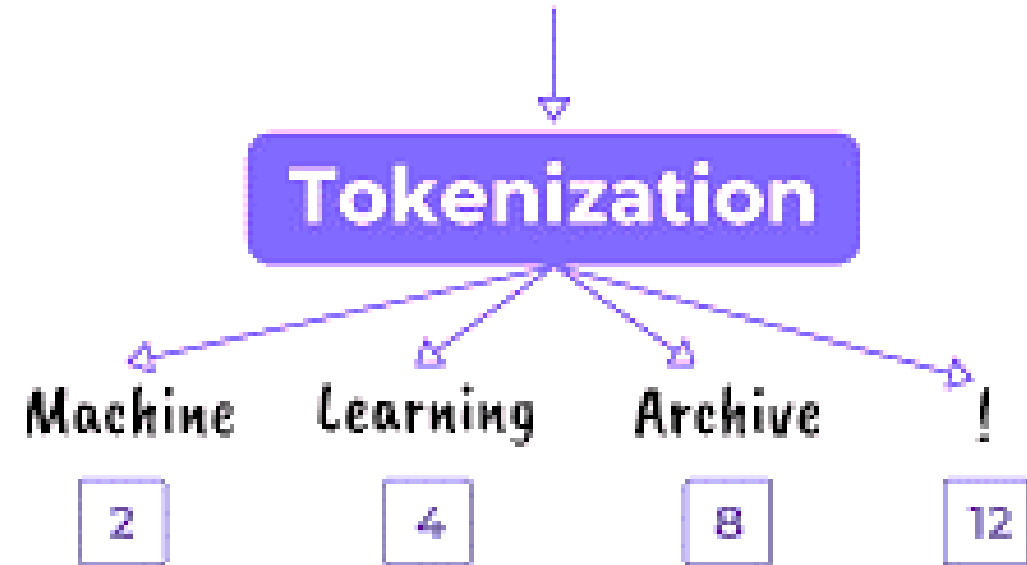
Splits text into individual words

Example:

"NLP is fun!" → ['NLP', 'is', 'fun']

Tools: nltk.word_tokenize, spaCy, etc.

"Machine Learning Archive!"



2. Sentence Tokenization

Splits text into individual sentences

Example:

"Hello. How are you?" → ['Hello.', 'How are you?']

Useful for tasks like summarization or document classification

Tokenization — Word, Sentence, and Subword Tokenization

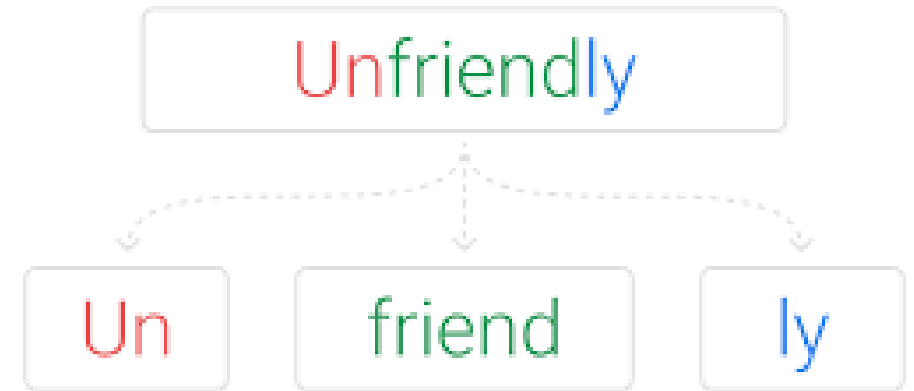
3. Subword Tokenization

- Splits words into meaningful subunits (subwords)
- Handles rare words or morphologically complex languages

Example:

"unhappiness" → ['un', 'happi', 'ness']

Tools: Byte Pair Encoding (BPE), WordPiece, SentencePiece (used in BERT, GPT, etc.)



Importance:

- Accurate tokenization improves model performance
- Choice of method depends on the task and language

Stemming and Lemmatization

Overview

Both techniques reduce words to their root forms, but they differ in method and accuracy.

1. Stemming

- Removes suffixes/prefixes to get the root form
- May produce non-dictionary words

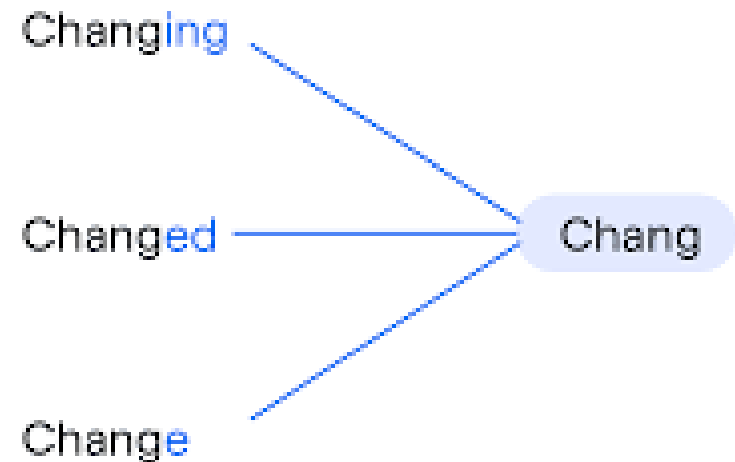
Example:

- "playing", "played", "plays" → "play"
- "studies" → "studi"

Tools: PorterStemmer, SnowballStemmer (NLTK)

- Faster but less accurate

Stemming



Stemming and Lemmatization

2. Lemmatization

- Converts words to their base or dictionary form (lemma)
- Requires context (POS tagging)

Example:

"running" → "run"

"better" → "good"

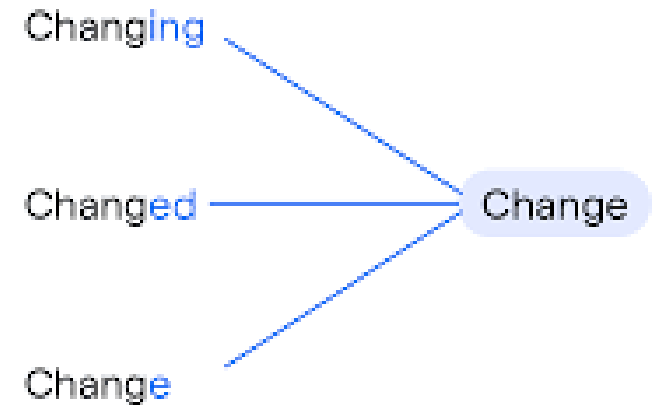
Tools: WordNetLemmatizer (NLTK), spaCy

- Slower but more accurate

Use Case:

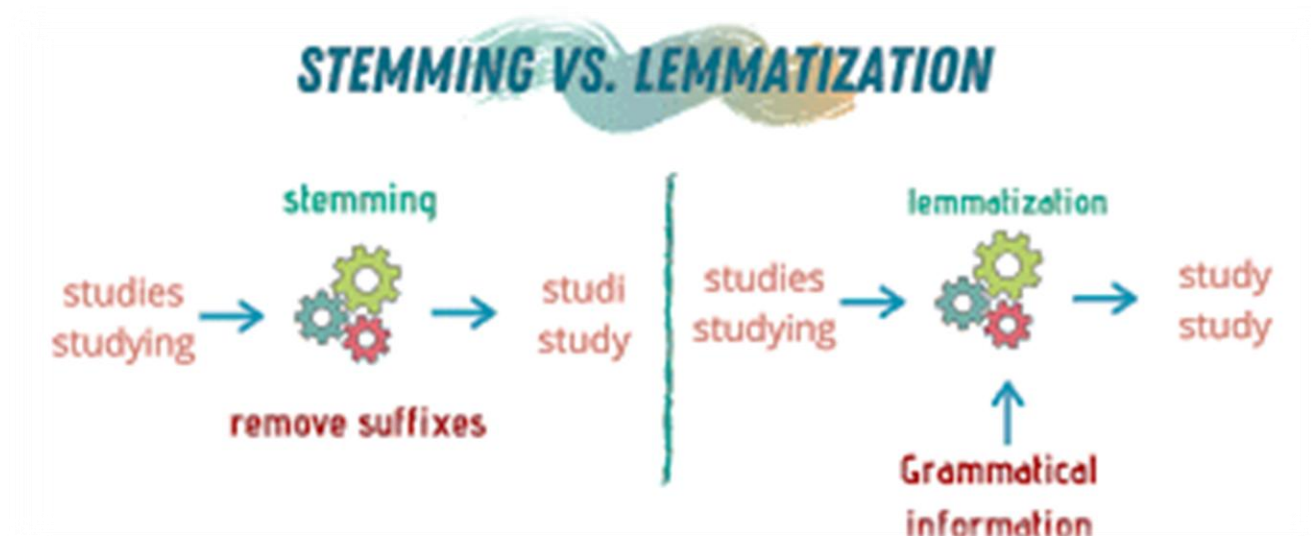
Choose stemming for speed-sensitive tasks, and lemmatization for accuracy-sensitive NLP tasks.

Lemmatization



Stemming and Lemmatization

Feature	Stemming	Lemmatization
Output	Root (may be invalid)	Dictionary word
Accuracy	Lower	Higher
Speed	Faster	Slower
Context needed	No	Yes



Stop Word Removal

What are Stop Words?

- Commonly used words in a language that carry little meaningful information
- Examples: "the", "is", "in", "and", "to", "of"

Stop Words

Why Remove Stop Words?

- Reduce noise in text data
- Improve performance of NLP models by focusing on important words
- Reduce dimensionality of data

[**“This”**, **“is”**, **“a”**, **“test”**]

Stop Word Removal

How to Remove Stop Words?

- Use predefined stop word lists available in NLP libraries like NLTK, spaCy
- Customize stop word lists based on domain or task

Example:

Original sentence:

"The quick brown fox jumps over the lazy dog."

After stop word removal:

"quick brown fox jumps lazy dog."



Considerations:

- Sometimes stop words carry important context (e.g., negations like "not")
- Always evaluate impact on your specific task before removal

Common Tools:

- NLTK's stopwords corpus
- spaCy's default stop words

Text Normalization

What is Text Normalization?

- Process of transforming text into a standard, consistent format
- Helps improve accuracy of NLP tasks by reducing variation

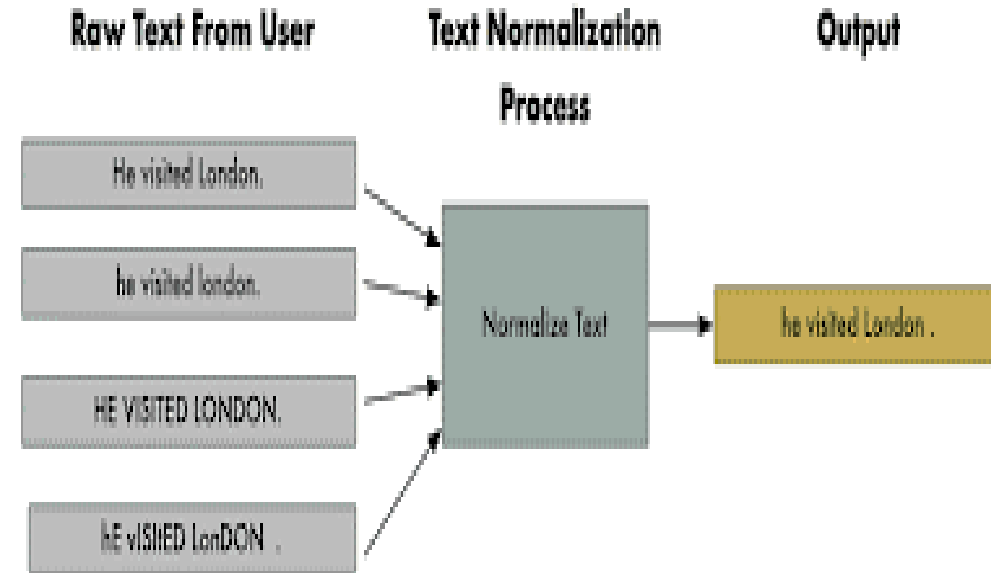
Key Aspects:

Handling Contractions: Expanding shortened forms

- e.g., "don't" → "do not"
- "I'm" → "I am"

Managing Slang and Informal Language

- e.g., "gonna" → "going to"
- "lol" → "laughing out loud"



Text Normalization

Why Normalize Text?

- Reduces vocabulary size
- Helps models better understand meaning
- Essential for noisy user-generated content (tweets, chats)

Techniques:

- Rule-based replacement dictionaries
- Regular expressions for pattern matching
- Pre-trained normalization models

Example:

Original: "I'm gonna grab some food, lol!"

Normalized: "I am going to grab some food, laughing out loud!"

Tools:

- Python libraries: contractions, textblob
- Custom mappings for domain-specific slang



Regular Expressions for Text Processing

What are Regular Expressions (Regex)?

- Powerful patterns used to search, match, and manipulate text
- Essential tool for text cleaning and preprocessing in NLP

Common Uses in NLP:

- Finding specific words or patterns
- Extracting email addresses, URLs, phone numbers
- Removing unwanted characters or symbols
- Splitting text into tokens based on patterns



Real-World Regex Applications



Regular Expressions for Text Processing

Basic Regex Syntax:

- . : Any character (a.b → acb, a9b, a-b)
- \d : Digit (0-9) (\d\d\d → 123, 231, 987)
- \w : Word character (letters, digits, underscore) (\w\w\w → abc, a98, Zb_)
- * : Zero or more repetitions (lo*l → ll, lol, loool)
- + : One or more repetitions (lo+l → lol, loooool)
- [] : Character set (e.g., [aeiou]) (gr[ea]y → gray, grey)
- ^ : Start of string (^Hello → Hello World)
- \$: End of string (end\$ → the very end)

[^]*?@[^]*?\. [^]*

Example:

Pattern to find all email addresses:

```
\b[A-Za-z0-9._%+~]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b
```

Remove all digits from text:

```
re.sub(r'\d+', '', text)
```

Regular Expressions for Text Processing

Python Regex Module:

```
import re
```

Functions: `re.search()`, `re.match()`, `re.findall()`, `re.sub()`

Why Use Regex?

- Flexible and efficient text pattern matching
- Speeds up preprocessing steps for NLP pipelines





**THANK
YOU**